

Software Requirements Document

1. Product Perspective

The Fantasy Premier League Pro (FPL Pro) is a standalone web application designed to serve as a comprehensive simulation platform for football management. It integrates real-world football data into a virtual environment by connecting to the official Fantasy Premier League (FPL) API, which allows the system to retrieve real-life match statistics, player information, and game week results.

The software is developed as a technical project that emphasizes modular architecture, utilizing a Python-based backend and a React frontend. Beyond its functionality as a simulation tool, the system is specifically structured to demonstrate the practical application of multiple software design patterns, ensuring the architecture is maintainable, extensible, and follows clean code practices. While it manages core fantasy mechanics like team building and transfers, it operates independently of live match streaming or external payment systems.

2. Product Functions

The Fantasy Tactics League platform provides the following core functions:

- **Team Management:** Users can create and manage virtual football teams composed of real Premier League players. This includes building squads within a fixed virtual budget and adhering to specific squad size constraints.
- **Transfer System:** The system enables users to perform player transfers between match weeks. This allows for strategic squad improvements while remaining within the defined budget limits.
- **Automated Scoring:** The software automatically calculates weekly points based on the real-life performances and match statistics of the selected players in actual Premier League fixtures.
- **Captain Selection:** Users can designate a specific player as the team captain for each match week to earn bonus points (typically doubled) based on that player's performance.
- **League Ranking and Standing:** The system provides a leaderboard functionality that sorts and ranks users based on their accumulated weekly points, allowing for comparison and competition between participants.

3. User Characteristics

The expected users of the Fantasy Tactics League platform and their respective skill levels are categorized as follows:

- **Football Fans:** Individuals who actively follow the English Premier League and possess a basic understanding of football rules and match mechanics.
- **Strategic Managers:** Users who enjoy strategic simulations and are capable of performing data-driven decision-making, such as budget management and long-term squad planning.
- **Academic Reviewers:** Instructors and students who will evaluate the system's underlying architecture, focusing on the correct application of Software Design Patterns and modular code.

practices.

4. Constraints

The development and operation of the Fantasy Tactics League platform are subject to the following technical and project-related constraints:

- **Budget Rules:** Users are required to build and maintain their fantasy squads within a fixed virtual budget of £100.0m.
- **Squad Composition:** Teams must adhere to valid football formations, consisting of 11 players with specific positional requirements (e.g., 1 Goalkeeper, 3–5 Defenders, 3–5 Midfielders, and 1–3 Forwards).
- **API Usage:** To respect the rate limits of the external football-data.org API, the system must utilize data fetching scripts or cached data to ensure stability.
- **Design Pattern Implementation:** The software architecture must explicitly implement and document at least four distinct software design patterns, such as Factory Method, Strategy, Observer, or Singleton.
- **Implementation Deadline:** All project deliverables, including the functional software and documentation, must be finalized and submitted by early April 2026.
- **Exclusions from Scope:** The system will not support real-time match streaming, financial payment systems, mobile application platforms, or advanced AI-based prediction features.

5. System Features (Use Case Based)

Use Case Diagram

The system's functionality is centered on the interaction between the User (the fantasy manager) and the System, which integrates data from an external Football Data API. The diagram depicts the User performing core management tasks such as creating teams, transferring players, and selecting captains, while the System handles automated point calculations and ranking updates based on real-world match data.

Detailed Use Case Descriptions

Use Case 1: Create New Fantasy Team

- **Title:** Create New Fantasy Team
- **Main Actor:** User
- **Goal:** To select an initial squad of 15 players within the defined budget and squad rules (positional and club limits) to participate in the league.
- **Preconditions:**
 1. The user has accessed the application and is authenticated.
 2. The user does not already have an active saved team.
 3. Real-world player data has been successfully fetched from the external API via the **APIService (Singleton)**.
- **Main Flow:**
 1. User navigates to the "Team Creation" interface.
 2. System displays a list of available Premier League players, with each player instantiated as a concrete object via the **PlayerFactory**.
 3. User selects players to fill the required squad positions (2 GKs, 5 DEFs, 5 MIDs, 3 FWDs).
 4. System updates the "Remaining Budget" and squad count in real-time as players are added or removed, utilizing the **RealTimeUIUpdater (Observer Pattern)** for instant feedback.
 5. User completes the squad and clicks "Save Team".
 6. System validates the squad against the £150.0m budget, the squad size (15), positional requirements, and team limits (max 3 per club) using the **ValidationStrategy (Strategy Pattern)**.
 7. System saves the team to the database via the **DatabaseManager (Singleton)**.
- **Postconditions:** A new fantasy team record is created, and the user is added to the league ranking system.
- **Exceptions:**
 - **Budget Exceeded:** If the total cost exceeds the £150.0m fixed budget, the system prevents saving and displays an error message.
 - **Invalid Squad Size:** The system blocks saving if the user has not selected the full required number of 15 players.

– **Validation Violation:** The system rejects saving if either the positional breakdown is incorrect (e.g., too many forwards) or if the user has selected more than 3 players from the same club.

This process is illustrated in the following image 1.

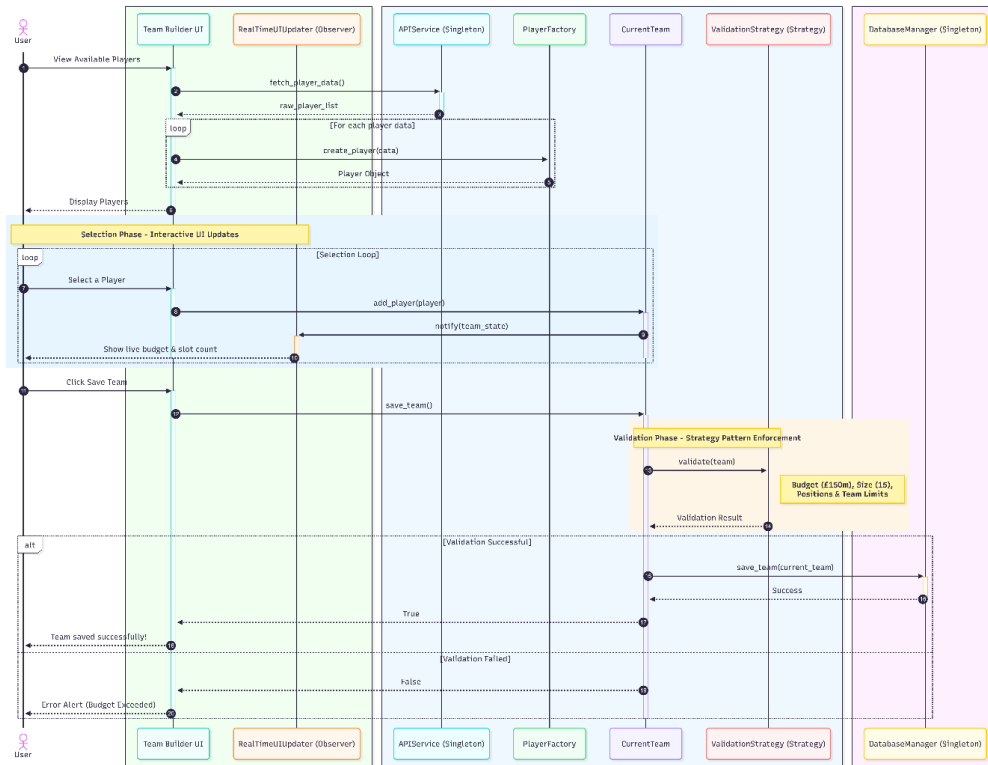


Image 1

Use Case 2: Point Calculation

- **Title:** Fantasy Point Calculation Engine
- **Main Actor:** System (Automated)
- **Goal:** To calculate the total points earned by a fantasy squad based on real-world player performance and tactical settings (Captain/Bench).
- **Preconditions:**
 1. A valid team has been successfully saved in the database.
 2. Match statistics (minutes, goals, assists, clean sheets, etc.) have been imported for the gameweek.
- **Main Flow:**
 1. System retrieves the current squad composition for the calculation.
 2. **TeamPointCalculator** identifies each player's position and assigns the correct **ScoringStrategy** (e.g., Goalkeeper, Defender).
 3. System calculates base points based on minutes played (1 or 2 pts), disciplinary marks (Yellow/Red cards), and bonus points.
 4. System applies positional scoring (e.g., +6 for Defender goals, +4 for Goalkeeper clean sheets) according to the strategy.
 5. System calculates advanced tactical bonuses such as the ICT Index benchmarks (MID $\geq$ 8.0, FWD $\geq$ 10.0) or Away Clean Sheets.
 6. The **ModifierStrategy (Decorator Pattern)** applies multipliers for the Captain (2x) and Bench (0x) players.
 7. System aggregates all individual scores and returns the total squad point tally.
- **Postconditions:** The squad's total score is successfully updated in the database for display and league rankings.
- **Exceptions:**
 - **Incomplete Data:** If match statistics for a player are entirely missing, the system awards 0 points and logs a warning.
 - **Invalid Position Strategy:** The system defaults to standard scoring (0 pts) if a player's position does not match a known strategy.

This process is illustrated in the following image 2.

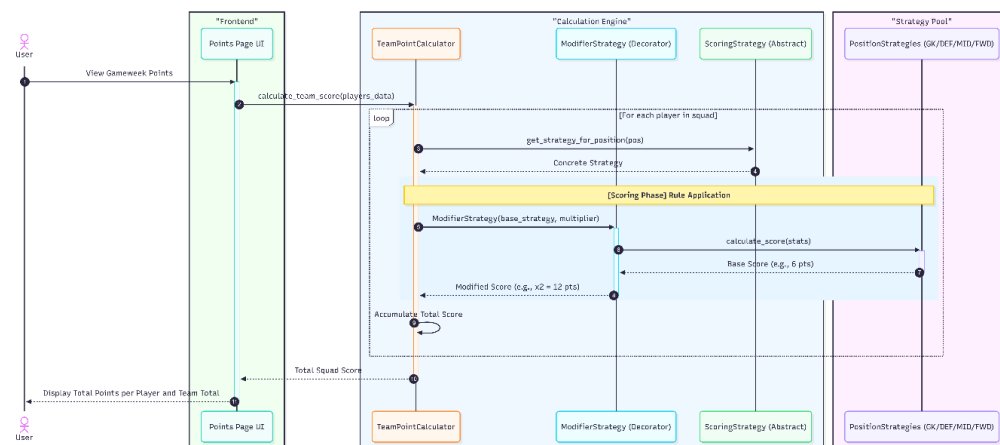


Image 2

Use Case 3: View League Leaderboard

- **Title:** View League Leaderboard
- **Main Actor:** User
- **Goal:** To view the current league standings and compare team performance against other participants based on total accumulated points.
- **Preconditions:**
 1. The system has completed at least one weekly point calculation using real match data.
 2. At least one user team and its score are registered in the **GlobalLeaderboard** cache.
- **Main Flow:**
 1. User navigates to the "League Table" or "Leaderboard" screen, triggering a GET request to the **/leaderboard** API endpoint.
 2. The API router delegates the request to the **GlobalLeaderboard** instance, which operates as a **Singleton** — ensuring a single, consistent ranking authority throughout the application.
 3. **GlobalLeaderboard** reads all team entries from its internal **_teams_cache**, which holds each team's name, total score, and remaining budget.
 4. Teams are sorted in descending order of accumulated points; ties are broken by remaining budget (higher budget ranks first).
 5. The sorted leaderboard is published to the **EventBus** via the **LEADERBOARD_REFRESHED** event, and all registered observers are notified: the **ConsoleNotifierObserver** prints the top-5 standings to the server log, and the **UIEventNotifierObserver** triggers a socket push to connected clients (**Observer Pattern**).
 6. System returns the ranked list — containing Rank, Team Name, Total Points, and Remaining Budget — to be displayed in the frontend.
- **Postconditions:** The user is presented with the most current, sorted standings of the league, reflecting all completed gameweek scores.
- **Exceptions:**
 - **Empty Leaderboard:** If no teams have been registered or scored yet, the system returns an empty list without raising an error.
 - **Score Not Yet Updated:** If a team's score has not changed since the last notification, the observer notification and EventBus publish are skipped to avoid redundant updates.

This process is illustrated in the following image 3.

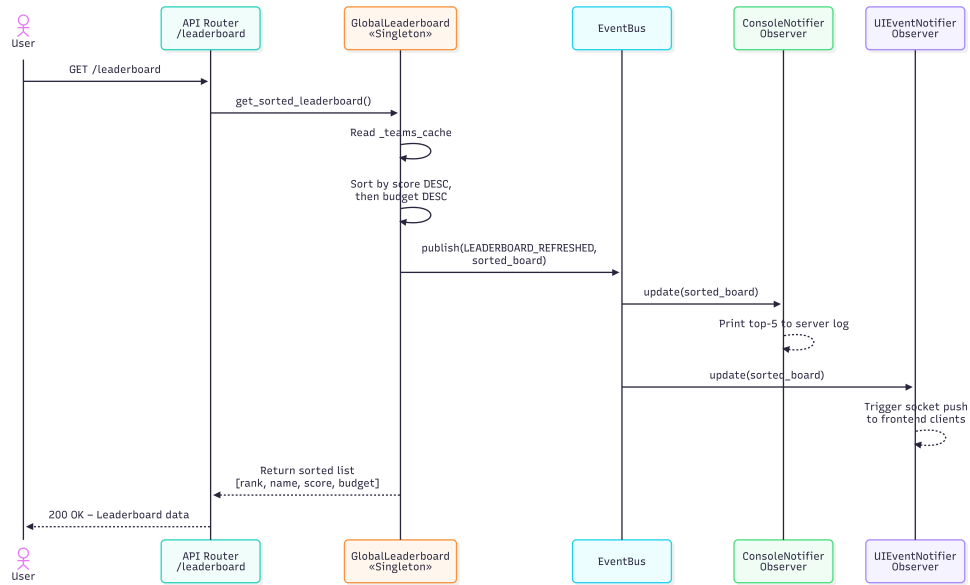


Image 3

Use Case 4: Fetch and Process Gameweek Data

- **Title:** Fetch and Process Gameweek Data
- **Main Actor:** System Administrator (Automated)
- **Goal:** To simulate and persist new gameweek performance statistics for all players, and to automatically trigger recalculation of all team scores upon data availability.
- **Preconditions:**
 1. A previous gameweek's player statistics already exist in the **stats** table of the database and serve as the baseline for the new gameweek.
 2. Position-based scoring strategies (Goalkeeper, Defender, Midfielder, Forward) are fully configured and available.
 3. The **TeamScoreUpdaterObserver** is registered as a subscriber to the **GAMEWEEK_DATA_FETCHED** event on the **EventBus**.
- **Main Flow:**
 1. The administrator triggers the **POST /admin/fetch-gameweek/{gameweek_id}** API endpoint to initiate a new gameweek data cycle.
 2. The API router delegates the heavy database operations to a **BackgroundTask** to avoid blocking the main API thread, returning an immediate HTTP 202 Accepted response.
 3. **GameweekDataFetcher** connects to the database and retrieves all player records from the **stats** table for the most recent existing gameweek, joining with the **players** table to obtain each player's position.
 4. For each player record, the system applies randomized statistical variations to simulate new match outcomes: goals and assists are varied with 15% probability, clean sheets and saves with 30% probability, and disciplinary events (yellow/red cards) with 10% probability. The ICT index is also adjusted with a random offset.
 5. The system selects the appropriate **ScoringStrategy** for each player based on their **position_id** (1=Goalkeeper, 2=Defender, 3=Midfielder, 4=Forward), using the **Strategy Pattern** to encapsulate position-specific scoring logic.
 6. Each player's **total_points** for the new gameweek is calculated by calling **strategy.calculate_score(row_dict)** with the updated statistics.
 7. All new player records are batch-inserted into the **stats** table in a single database transaction. If any error occurs, the transaction is rolled back.
 8. Upon successful persistence, the system publishes the **GAMEWEEK_DATA_FETCHED** event via the **EventBus**, passing the new gameweek ID.
 9. **TeamScoreUpdaterObserver** receives the event (**Observer Pattern**) and queries the database for all registered user teams and their player compositions for the new gameweek.
 10. For each team, **TeamPointCalculator** aggregates individual player scores (from Use Case 2 logic) into a single team total.
 11. The system publishes a **TEAM_SCORES_UPDATED** event for each team via the **EventBus**, carrying the team ID, name, new score, and budget. This event triggers the **GlobalLeaderboard** to refresh its rankings (Use Case 3).
- **Postconditions:** New gameweek statistics are persisted to the database, all team scores are recalculated, and the leaderboard is automatically refreshed to reflect the latest standings.
- **Exceptions:**

- **Database Error:** If the batch insert into the **stats** table fails, the database transaction is rolled back and an exception is raised, preventing partial data from corrupting the dataset.
- **Missing Player Stats:** If a player has no stats entry for the new gameweek (e.g., did not play), the **TeamPointCalculator** awards 0 points for that player by default.
- **Unknown Position Strategy:** If a player's **position_id** does not match any known strategy, the system defaults to the **MidfielderScoringStrategy** as a safe fallback.

This process is illustrated in the following image 4.

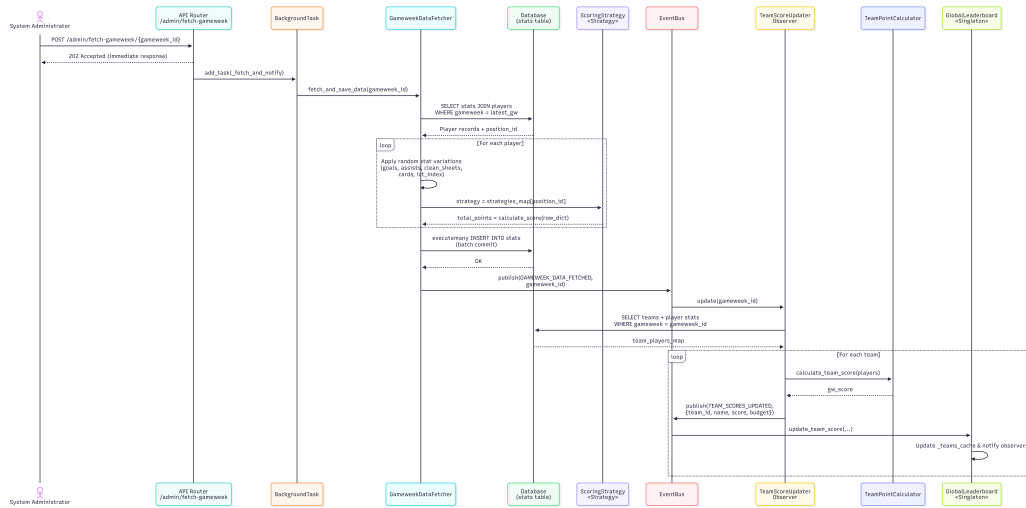


Image 4

6. Non-Functional Requirements

This section describes the system qualities and constraints that ensure the platform operates effectively beyond its core functionalities.

6.1 Usability

- **Intuitive Interface:** The "Team Builder" and dashboard must be designed so that football fans can manage teams and transfers without technical difficulty.
- **Visual Feedback:** The system must provide immediate visual feedback, such as highlighting negative budget balances in red, to help users avoid input errors.
- **Efficient Selection:** The player search and filtering tools (by name, team, or position) should allow users to quickly find and select players.

6.2 Performance

- **API Efficiency:** The response time for loading the player list (containing 800+ items) from the API must be under 1 second to ensure a smooth user experience.
- **Processing Speed:** Weekly point calculations for all users should be completed within seconds after fetching the latest match data.
- **Data Flow:** The integration with the external football-data.org API must be optimized to ensure consistent data flow without lagging the application.

6.3 Portability

- **Cross-Platform Support:** The system, built with a Python backend and React frontend, must be deployable on Windows, Linux, and macOS.
- **Browser Compatibility:** The application must function consistently across all modern web browsers.

6.4 Reliability

- **Deterministic Scoring:** The scoring logic must be stable and consistent; running the calculation for the same match data must always produce the same points.
- **Failure Management:** The system must handle API failures gracefully, such as by displaying cached or mock data if the live API service is unavailable.
- **Architectural Integrity:** The use of modular design patterns must ensure that the system remains maintainable and extensible throughout its lifecycle.

7. Software Interfaces

The Fantasy Tactics League application interacts with various external software components and systems to ensure data persistence and real-time updates:

- **Football Data API:** The system serves as a client to the football-data.org API to fetch Premier League-specific data, including player metadata, match results, and fixtures. This integration is critical for the automated scoring and ranking functions.
- **External Data Format:** All information retrieved from the external API is consumed and processed in JSON format to be mapped into the system's internal objects.
- **Database Management System:** A local database (such as SQLite) is utilized to store persistent application data, including user squad configurations, historical point totals, and player price histories.
- **Version Control:** GitHub is used as the primary software interface for collaborative development, source code versioning, and tracking team contributions through issues and pull requests.
- **Architectural Documentation Tools:** The system's design patterns and UML diagrams are developed and maintained using specialized software interfaces such as Draw.io or Lucidchart.